

RÉPUBLIQUE DU CAMEROUN

PAIX-TRAVAIL-PATRIE

SAINT JEAN INGÉNIEUR

REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

SAINT JEAN ENGINEER



INSTITUT UNIVERSITAIRE SAINT JEAN
SAINT JEAN INGÉNIEUR

PROJET DE SIMULATION DE RESEAU SIMNet

RAPPORT TECHNIQUE

Membres du groupe 5 de Programmation Orientée Objet en Python:

- FOTSO Verdiane
- NJOYA Ariel
- SEUMO Yann
- NTANDZI Claude Manuella
- NINKAM Noémie

Sous la supervision de:

M. Stephane FEDIM

Année académique 2025-2026

Introduction

Dans le cadre de la validation de l'unité d'enseignement Programmation Orientée Objet en Python, nous sommes chargés, en groupe de 5, de concevoir et développer SIMNet : un simulateur de réseau d'entreprise entièrement orienté objet. Ce projet nous a permis de mettre en pratique les concepts fondamentaux de la POO encapsulation, héritage, abstraction et polymorphisme – à travers la réalisation d'un outil complet de modélisation et de simulation réseau. Le simulateur doit être capable de représenter des équipements (routeurs, switches, serveurs, firewalls, points d'accès Wi-Fi et terminaux clients), de les relier par des liens caractérisés par une bande passante et une latence, et de faire circuler des paquets selon des règles de routage et de sécurité. Le présent rapport décrit l'architecture logicielle retenue, le diagramme de classes UML, les choix de conception et les principales difficultés rencontrées. Notre équipe a adopté une méthodologie itérative avec un suivi rigoureux via Git, garantissant une intégration progressive et une qualité de code homogène.

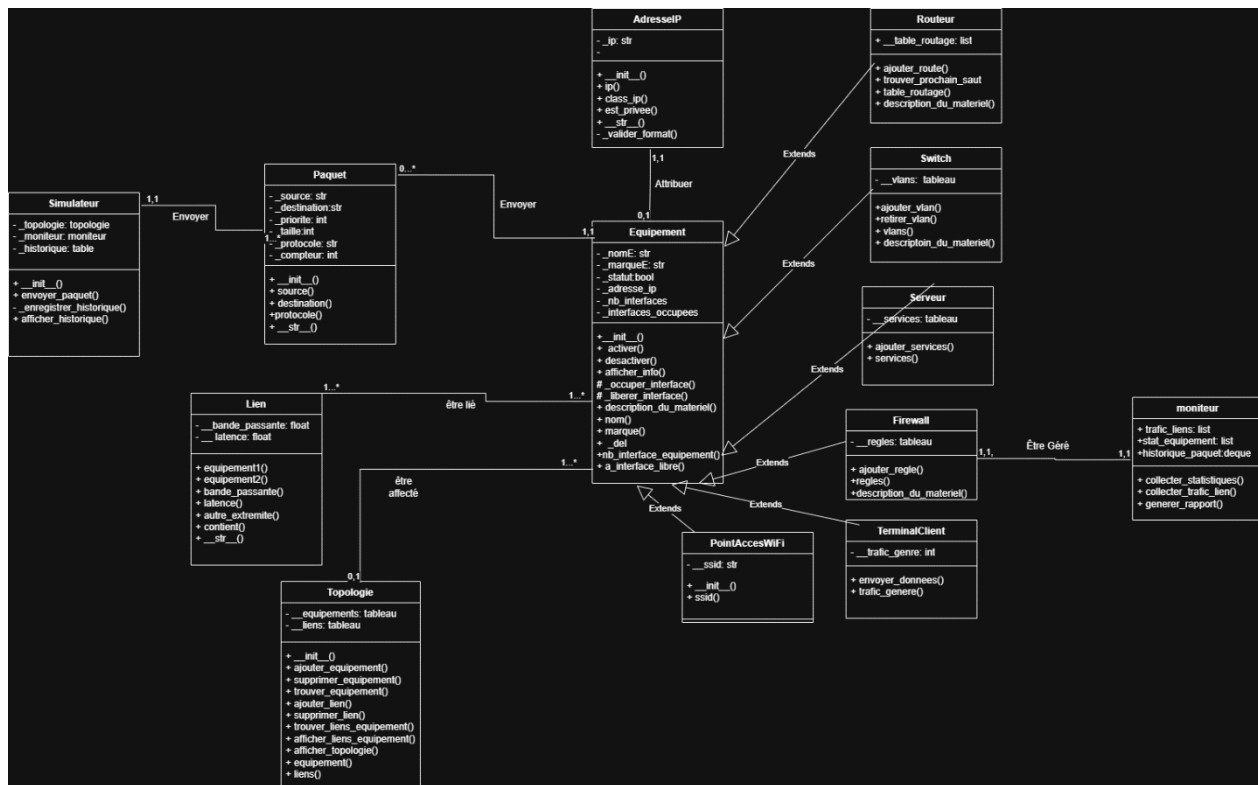
I. Présentation du groupe et des rôles

Pour la réalisation du projet, nous avons pu faire un groupe par affinité. Les étudiants qui constituent les membres du groupe sont : **Noémie NINKAM**, **Ariel NDOJOYA**, **Verdiane FOTSO**, **Manuella NTANDZI**, **Yann SEUMO**.

La répartition des rôles à été faite par Module.

- **Le Module 1** : a été géré par **Yann SEUMO**, il était donc en charge de la création des **classes des équipements**, la gestion des **liens** et la modélisation de la **topologie**.
- **Le Module 2** : a été géré par **Ariel NDOJOYA**. Il était responsable de l'implémentation de la **circulation des paquets** entre équipements, le routage, et les statistiques associées. Il avait donc pour principal tâche, La création de la classe **Paquet** pour les paquets réseau, la création de la classe **Simulation** pour simuler le transfert des données.
- **Le module 3** : a été géré par **Manuella NTANDZI** qui a implémenté le **firewall**, l'authentification, les règles de filtrage, la journalisation. Elle a principalement créé la classe **Règle** et ses méthodes pour la gestion des règles de filtrage avec une action (AUTORISER ou BLOQUER).
- **Le Module 4** : a été géré par **Verdiane FOTSO**. Elle était en charge de **collecter les métriques** en continu, **tenir un historique**, et **générer un rapport d'exploitation**. Elle a principalement créé la classe **moniteur** qui permet la collecte des statistiques, le calcul du taux d'utilisation des liens, la génération du rapport.
- **Le Module 5** : a été géré par **Noémie NINKAM**. Elle était en charge de concevoir et implémenter un **menu textuel** permettant à l'utilisateur de piloter toutes les fonctionnalités du simulateur sans modifier le code.

II. Diagramme de classes complet



III. Justification des choix de conception

1. Choix de conception

Pour créer notre **simulateur SIMNet**, nous avons utilisé la Programmation Orientée Objet. Cela nous permet de découper le réseau en plusieurs blocs faciles à gérer.

a) L'Héritage (Pour éviter de se répéter)

Dans un vrai réseau, un routeur ou un switch restent avant tout des équipements : ils ont tous un nom, une adresse IP et un statut (actif/inactif).

- **Dans notre code :** On a créé une classe générale Equipement. Ensuite, les classes Routeur ou Commutateur héritent de cette classe.
- **Pourquoi ?** Ça nous évite de réécrire les mêmes lignes de code pour chaque appareil. Tout ce qui est commun est écrit une seule fois dans Equipement.

b) L'Encapsulation (Pour protéger le code)

L'encapsulation, c'est comme mettre une barrière de sécurité autour des informations importantes d'un objet pour que les autres composants ne fassent pas n'importe quoi avec.

- **Dans notre code :** Le statut de l'appareil (`_statut`) est caché et protégé à l'intérieur de l'équipement.
- **Pourquoi ?** Ça empêche un autre fichier de modifier directement ou par erreur l'état d'un routeur. Si on veut lire ou changer le statut, on est obligé de passer par une fonction bien précise (le Getter `.statut`). Cela protège la cohérence de notre simulation.

c) L'Abstraction (Pour faire simple)

L'abstraction consiste à masquer les détails techniques compliqués pour ne montrer que ce qui est utile.

- **Dans notre code :** Quand on manipule un Paquet, on s'intéresse juste à sa destination et à son contenu. On n'a pas besoin de voir toute la mécanique interne des octets qui bougent à chaque instant.
- **Pourquoi ?** Ça rend le code beaucoup plus lisible et plus facile à comprendre pour quelqu'un qui lit notre projet.

d) Le rôle du Moniteur (La séparation du travail)

Pour que le projet soit propre, chaque fichier a un seul travail bien précis.

- **Dans notre code :** Le fichier `Moniteur.py` ne s'occupe pas de faire circuler les données ou de connecter les câbles. Son unique rôle est de regarder la simulation se faire, de compter les paquets perdus ou transmis, et de créer le fichier `rapport_simnet.txt` à la fin.
- **Pourquoi ?** Si demain on veut changer la présentation du rapport ou ajouter de nouvelles statistiques, on modifie uniquement `Moniteur.py`. On est sûr que cela ne va pas dérégler ou casser le reste du réseau.

2. INTERACTIONS ET COMMUNICATIONS ENTRE LES CLASSES

Pour que le simulateur fonctionne, les classes ne travaillent pas seules dans leur coin : elles s'utilisent les unes les autres de manière logique pour reproduire le comportement d'un vrai réseau.

a) La classe Topologie utilise la classe Equipement

- **La classe Topologie** contient une liste qui stocke tous les objets de type Equipement (les routeurs, les commutateurs, etc.).
- **Justification** : Une topologie réseau, c'est comme une carte géographique : elle a besoin de connaître la liste exacte de toutes les machines présentes sur le terrain et comment elles sont reliées pour pouvoir gérer le réseau global.

b) La classe Moniteur utilise la classe Topologie

- Dans sa **fonction generer_rapport**(self, topologie), la classe Moniteur reçoit toute la topologie en paramètre pour aller lire la liste des équipements (topologie.equipements).
- **Justification** : Le Moniteur est un outil de surveillance externe. Pour faire son rapport et afficher le statut de chaque appareil (ACTIF ou INACTIF), il a besoin que la topologie lui ouvre ses portes et lui montre la liste des machines à inspecter.

c) Les méthodes de transmission utilisent la classe Paquet

- Les fonctions comme **envoyer_paquet()** ou **recevoir_paquet()** dans les équipements prennent un objet Paquet comme argument.
- **Justification** : Le Paquet est l'unité d'information qui circule. Passer l'objet Paquet de fonction en fonction permet de simuler le voyage réel d'une donnée de câble en câble, en gardant ses informations (source, destination, contenu).

d) Le Moniteur utilise la fonction de datation : datetime

- Lors de l'écriture du rapport, le moniteur appelle **datetime.now()**.
- **Justification** : Un rapport technique n'a aucune valeur s'il n'est pas daté. Cela permet de marquer précisément l'instant de la simulation (jour, heure, minute, seconde) pour que l'administrateur réseau sache exactement quand les statistiques ont été récoltées.

IV. Difficultés rencontrées et solutions apportées

Nous avons rencontré plusieurs problèmes notamment :

- **L'utilisation de git** : Nous avons dû nous familiariser avec l'environnement **git** et la plateforme **github** pour éviter les conflits. Cependant, nous avons rencontré un problème de conflits dans nos travaux.

- **La compréhension du projet :** Nous avons pris plusieurs jours pour la compréhension et la modélisation de nos classes et méthodes avant de commencer le code
- **L'implémentation de la sécurité avec le parefeu ;**

Conclusion

Le projet **SIMNet** a été mené à bien par notre groupe dans le temps imparti. Tous les modules fonctionnels attendus sont opérationnels : la topologie permet d'ajouter, de supprimer et de connecter des équipements en gérant le nombre limité d'interfaces ; le simulateur de trafic détermine des chemins, transmet les paquets saut par saut et comptabilise les statistiques ; **le firewall** intègre une authentification administrateur, des règles de filtrage flexibles et une journalisation horodatée ; **le moniteur** collecte les métriques et génère un rapport d'exploitation complet ; enfin, **l'interface console** interactive offre un accès convivial à toutes ces fonctionnalités. Les choix d'architecture : **classe abstraite** (Équipement), **héritage** pour les équipements spécifiques, **encapsulation stricte** des attributs, et **séparation des responsabilités** (topologie, paquets, sécurité, surveillance) ont permis d'obtenir un code maintenable, évolutif et conforme aux principes de la POO. Les tests unitaires et d'intégration ont validé le comportement attendu, et l'utilisation de **Git** avec des commits réguliers a facilité la collaboration en groupe. Les perspectives d'amélioration une gestion plus fine des masques réseau et des plages de ports dans le firewall, ainsi qu'une interface graphique (GUI). Néanmoins, la version actuelle constitue une base solide et démontre notre maîtrise des concepts enseignés. Nous tenons à remercier l'examineur pour l'encadrement et pour l'opportunité de réaliser ce projet fédérateur.